BAC Project & Tooling Guide

Build a CubeSat – project, software and tooling standard. Version 0.15, 2026-10-27

This document defines engineering, software, repository, data, and implementation conventions used across the BAC project. It is the primary reference for LLMs and human contributors when creating or updating project tooling and technical workflows. The goal is consistency, not uniformity – each tool context has its own implementation constraints, but project conventions remain predictable and maintainable.

Relationship to the BAC Identity and Interface guides. The BAC Identity Guide is authoritative for brand and communication identity, canonical visual tokens, typography, imagery, and Ops/Mission modes. The BAC Interface Design Guide is authoritative for interface behavior, semantic state application, accessibility, components, data visualization, feedback and progress, terminal presentation, and physical indicators. This Project & Tooling Guide is authoritative for licensing, versioning, code and file conventions, URLs and repositories, data formats, SKUs, time reference, CLI implementation, config and secrets, marimo implementation, Blender extensions, and documentation process. Where implementation needs a visual or interface value, reference the upstream guide rather than creating a local variant. The BAC Trademark Policy governs how others may use the BAC name and marks; this guide's licensing table (Section 2.1) points to it.

1. Core Values

These apply across all tool types.

- **Minimal surface, maximum clarity.** Show only what the user needs at each step. Suppress noise by default; surface detail on request or on failure.
- **Fail loudly, succeed quietly.** Errors must be visible and actionable. Successful routine operations need no ceremony.
- **Engineering aesthetic.** Practical over flashy. Visual treatment follows the BAC Identity Guide and interaction treatment follows the BAC Interface Design Guide.
- **Open-source first.** Hardware, software and documentation are open; the BAC name and marks are not (Section 2.1). Human-readable files and formats wherever practical. Avoid unnecessary complexity. Favor maintainability and portability.
- **No emojis.** Symbols are used only where they carry unambiguous semantic meaning: ✓ for pass, × for fail, ! for warning, → for progression. Decorative emojis are never used.

2. Project-Wide Conventions

2.1 Licensing

Every BAC artifact is licensed as follows. Do not deviate.

Artifact type	License
Hardware (schematics, PCB layouts, mechanical)	CERN OHL-S 2.0

Artifact type	License
Software	MIT License
Documentation	CC BY-SA 4.0
Brand marks (name, logo, wordmark, lockup)	Not openly licensed – BAC Trademark Policy

The BAC Marks – the name Build a CubeSat, the abbreviation BAC, the cube mark, the wordmark and their lockup – are excluded from all three licences, also where they appear inside an openly licensed repository or file; their use by others follows the BAC Trademark Policy (https://github.com/buildacubesat/bac-design/blob/main/guides/bac-trademark-policy.md; nihil.space GmbH, hello@buildacubesat.space). A repository that ships logo artwork says so in its README's licence section: "The Build a CubeSat name and logo are not covered by this repository's licences; see the BAC Trademark Policy."

Include the appropriate SPDX identifier in every pyproject.toml:

Ilicense = { text = "MIT" }

\2.2 Versioning

Physical and tangible items (PCBs, hardware assemblies, mechanical parts) use a human-friendly scheme:

Iv<version>r<revision>[.<patch>][-<variant>]

\

Examples: v1r2, v3r4.1, v2r2.1-flight. The version increments on significant redesigns; the revision on iterative changes; the optional patch on minor corrections; the variant suffix distinguishes build configurations, for example -flight or -proto.

Software uses Semantic Versioning 2.0.0: MAJOR.MINOR.PATCH, for example 1.2.3. In pyproject.toml and --version output, prefix with v: v1.2.3.

Git tags mark a tool version only where another repository pins it: <tool>-v<X.Y.Z> on the commit that produced the pinned output, for example bac-antenna-optimizer-v0.7.3, which bac-hardware's antenna.toml names. Branches merge into main by fast-forward, never squashed, so a tag stays on main's history.

Product, subsystem, and software documentation documents another part of the project and references that part's versioning scheme. The document title names the subject and its version; these docs do not carry a separate document version number. Examples: EPS v2r4 Readme, bac-kicad-generate-artifacts v0.3.1 User Guide.

Governance documents and standards are the exception. The BAC Identity Guide, BAC Interface Design Guide, BAC Project & Tooling Guide, and comparable project-wide standards are independently versioned because they govern multiple artifacts rather than documenting one versioned subject. Their version increments on substantive changes and each carries its own version history.

2.3 Naming and Identifiers

- Full name: Build a CubeSat
- Short form: BAC, always fully capitalized in display text and prose – never "Bac" or "BaC"
- In code and filenames: lowercase bac is fine, for example bac-toolname, bac_upload, bac-eps-bom.csv. Mixed case is never acceptable.
- Display naming and voice follow the BAC Identity Guide. Use the full name on first reference within a document or UI; the short form thereafter.
- Do not use "buildacubesat" as a display name. It is acceptable as a URL slug or code identifier only.
- Legal entity: nihil.space GmbH (DACH region), nihil.space LLC (international). Based in Bern, Switzerland.
- Attribution fields in files follow one rule with three cases (decided 2026-10-24). A field that asks for the legal entity – KiCad's title-block Company, the organization of a STEP header, 3MF Designer – carries nihil.space GmbH. A field that asks for the project or product – a title, a FILE_DESCRIPTION, a 3MF Title – carries Build a CubeSat (with the part and version where the field is per part). A single mixed field with no such distinction carries Build a CubeSat by nihil.space GmbH. The tools that write these fields (bac-kicad-maintain, bac-freecad-generate-artifacts) take the values from their config, so the rule lives in one place per machine; STL carries no metadata and is exempt.

2.4 URLs

Context	URL
Main project website	https://buildacubesat.space
User-facing links (docs, products, releases)	Use bac.page short links where available
API and machine-readable endpoints	Full canonical URL

When pointing users to documentation or external resources in tool output, use the bac.page short link if one exists.

2.5 Identity and Interface Dependencies

Do not redefine the BAC visual system in tooling code or project documentation.

- Canonical brand name and display rules, colors, typography, spacing, radii, shadows, imagery, and Ops/Mission communication modes live in the BAC Identity Guide.
- Interaction semantics, accessibility, light and dark behavior, components, data-visualization presentation, terminal presentation, progress and feedback, and physical-indicator semantics live in the BAC Interface Design Guide.
- Tool and platform implementations may carry literal values where an API requires them, for example Rich markup or a self-contained marimo style cell, but those copies are not authoritative and are not edited by hand. The executable source of the Identity Guide's tokens is tokens.json in bac-utils (tools/bac-design-tokens); its generator, bac-design-tokens, writes the reference gallery's and the bac-suite's tokens.css, the Rich colours in

bac_common.ui and every marimo style cell. To change a value, change tokens.json and the Identity Guide together, run uv run --no-sync bac-design-tokens, and commit the regenerated copies; bac-design-tokens --check fails CI when a copy is stale. A repository outside bac-utils, such as BAC Ground, takes a copy of the generated tokens.css.

- Terminal output inherits the user's terminal font and theme; do not attempt font-level styling or app-controlled light and dark mode.
- GUI and web tooling must support the Interface Guide's light and dark requirements and use the Identity Guide's semantic tokens.
- Host applications may require native presentation conventions. Two exceptions to BAC's general sentence-case display rule are documented: Blender's title-case labels (Section 5.1) and the title-case headings and callout titles of the marimo tools (Section 4.3, decided 2026-09-10). Both are recorded here so neither is mistaken for drift.

2.6 Glyphs and Technical Notation

- Use a period as the decimal separator.
- Use ' as the thousands separator. CHF and metric values use ' in the Swiss style: 12'059 CHF, 1'137.74 CHF. USD prices follow the American comma convention: $1,517.
- The thousands separator is a display convention only. Machine-readable output – CSV, JSON, TOML, and anything another tool parses – carries plain unformatted numbers.
- Use × for quantities and formulas, for example 2× INA260, 100×100×227 mm. Never a lowercase "x".
- Use of · as a spacer is encouraged where it makes sense. For numbered headings, always use a period: 1., 1.1, 2., 2.1.
- Never use an em dash (—), only the Gedankenstrich (–).
- Use of font ligatures is encouraged where appropriate.
- Axis directions are written with letters only: Xm, Xp, Ym, Yp, Zm, Zp. Never -X, +Y or any other sign notation. In command-line values and file names the letters are lowercase: xp, xm, yp, ym, zp, zm (--face zp). A tool may accept other spellings as aliases but prints the letter form.

2.7 Data Exchange Formats

Choose format based on intended use:

Format	Use when
CSV	Tabular data for both human inspection and machine processing. Default for BOM exports, telemetry logs, test results. Delimiter: ,. Encoding: UTF-8. Header row always present.
TO	Configuration files and parameter sets a human may edit by hand. Used for tool

Format	Use when
TOML	configs, notebook parameter snapshots, any structured human-edited input.
JSON	Machine-to-machine data exchange, API responses, structured outputs not intended for manual editing.
TSV	Avoid unless a source system produces it natively, for example a spreadsheet copy-paste. Prefer CSV with explicit quoting.
YAML	Avoid for new tooling. Acceptable in existing Zephyr and F´ configuration files where the framework requires it.

File naming convention:

I{project}-{descriptor}-{YYYY-MM-DD}.{ext}

\

Examples: bac-eps-bom-2026-06-04.csv, bac-hk-telemetry-2026-06-04.csv.

Do not encode version numbers in file names. Use a version field inside the file or rely on git history.

2.7.1 Mission Profile Contract

Tools that snapshot their parameters to TOML use the shared mission profile contract below. It is what lets one tool load a profile written by another, and it is the standard for notebook parameter snapshots referred to in Section 2.7. The marimo tools in Section 4 implement it; anything that has to exchange a mission case with them follows it too.

Top-level keys: name, a human-readable name for the mission case; tool, the writing tool as bac_<tool>; and tool_version, that tool's version.

The shared tables and their key names are normative. A tool that needs a value one of these already carries uses the name below rather than inventing its own, which is what keeps a fourth tool compatible by construction:

Table	Keys
[orbit]	altitude_km, inclination_deg, ltdn_hours, epoch, sim_days, min_elevation_deg
[ground_station]	station, latitude_deg, longitude_deg, min_elevation_deg
[target]	name, latitude_deg, longitude_deg
[map]	tiles, zoom, key
[spacecraft]	form_factor, size_x_cm, size_y_cm, size_z_cm, mass_kg, drag_coefficient, attitude_mission, attitude_after_mission
[[deployables]]	one table per deployable: name, shape (panel or cylinder), count, width_cm, length_cm, flow_angle_deg, deployed, deploy_at_end_of_mission

[spacecraft] describes the body and [[deployables]] what sticks out of it for drag; the orbital lifetime notebook writes both, the thermal, orbit viewer and power budget notebooks read them. Its attitude values – tumbling, min, max, ram_x, ram_y, ram_z – name an axis, not a direction,

because a box drags the same on either face. A tool that needs a pointing direction (the power budget's Sun or nadir pointing, the thermal tool's hot face) writes it in the §2.6 letter form in its own table, nadir_zm, sun_zp, and may take the drag attitude as a hint for its default. Solar panels stay in the power budget's own [[panels]] array (name, face, area_cm2, deployed), because a deployable is not always a panel and a panel not always a deployable; where one object is both, the two entries share a name and a reader matches them by it, as [[results.<tool>.modes]] rows are matched. [spacecraft] is not a signature table: a profile carrying it is attributed to its tool key.

Tool-specific tables follow the tool that owns them. Any key left out keeps the loading tool's default, and a tool ignores what it does not own rather than failing on it.

Results go in a [results.<tool>] table at the end, where <tool> is the slug that appears in the top-level tool key as bac_<tool>. It carries the headline figures a sibling may take and is ignored when the writing tool loads its own file back. Per-row results go in an array of tables under it, [[results.<tool>.modes]], one entry per row of the tool's mode table, so a sibling can match rows by name or modulation.

Authorship is detected from the tool key, and failing that from a signature table that only one tool writes: [[modes]], [sensor] or [storage]. Each tool states in its loader status block what it took from a foreign profile and what it ignored.

I```toml name = "Demo mission, 500 km SSO" tool = "bac_link_budget" tool_version = "0.7.0"

[orbit] altitude_km = 500 inclination_deg = 97.4 ltdn_hours = 10.5 epoch = "2026-09-14T00:00:00Z" sim_days = 7 min_elevation_deg = 10

[ground_station] station = "Bern" latitude_deg = 46.95 longitude_deg = 7.44 min_elevation_deg = 10

[target] name = "Bern" latitude_deg = 46.95 longitude_deg = 7.44

[map] tiles = "light_nolabels" zoom = 2 key = ""

[results.link_budget] # headline figures this tool publishes for its siblings; ignored on load

[[results.link_budget.modes]] # one table per row of the tool's mode table, matched by name or modulation

\

The BAC demo mission's planning orbit is a 500 km Sun-synchronous orbit (inclination 97.4°, LTDN 10:30); the BAC profiles shipped with the notebooks use it.

Reader and writer implementation for the marimo tools is in Section 4.7.

2.8 Repository Locations

The BAC project is split across two forges. Knowing which is which matters when writing tools that reference or interact with repos.

Codeberg is the primary home for all BAC project source:

| **Repo** | **Contents** |

|---|---|
codeberg.org/buildacubesat-project/bac-software	Flight software, firmware
codeberg.org/buildacubesat-project/bac-hardware	KiCad schematics, PCB layouts
codeberg.org/buildacubesat-project/bac-structure	Mechanical and structural files
codeberg.org/buildacubesat-project/bac-docs	Documentation source

GitHub is secondary, for projects that depend on GitHub-specific infrastructure such as GitHub Pages and GitHub Actions:

Repo	**Contents**
github.com/buildacubesat/CubeSat-Resources	CubeSat Resources website
github.com/buildacubesat/bac-page	URL shortener (GitHub Pages); bac.page until 2026-10, the old name redirects
github.com/buildacubesat/bac-utils	BAC tools, shared libraries, engineering notebooks, the operations suite, the Blender extension and the design tokens (tokens.json and bac-design-tokens; GitHub Actions CI)
github.com/buildacubesat/bac-design	The five BAC guides as Markdown with Word exports, and the brand assets; the normative text the tokens and tools follow

When a tool needs to reference a repo URL, link to Codeberg by default. Use GitHub URLs only for the four repos that are hosted there.

2.9 SKU Conventions

Tools that interact with the Shopify store, inventory, fulfillment, or any other part of the production pipeline must generate and parse SKUs according to this scheme:

Ibac-\<cat\>-\<sub\>-\<name-with-hyphens\>-\<versionstring\>\[-\<variant\>\]

\

Field	**Description**
bac	Mandatory prefix. Always lowercase.
cat	Category, for example dev, flight, tool, merch.
sub	Subcategory, for example kit, adcs, eps, shirt.
name	Short product name slug, lowercase, no spaces, for example main-board, interconnect, logo.
versionstring	Hardware versioning string per Section 2.2, for example v2r1, v3r4.1. Always present; SKUs are version-specific.
variant	Optional. Shop-specific differentiator: size (s, m, l, xl), color, or origin (eu for made in Europe). Omit if the product has no variants.

All fields are lowercase. Hyphens are the only separator. No spaces, underscores, or dots except within the versionstring patch field.

Examples:

Ibac-dev-adc-magnetorquer-XY-2U-v2r1.1

bac-edu-kit-1.5U-v2r4

bac-flight-kit-2U-v1r0-exolaunch

bac-merch-rbftag-logo-v1r0-m

bac-merch-shirt-galaxy-v1r0-eu

\

When a tool reads or writes SKUs, validate against this pattern before passing them to any API. A SKU that does not match the scheme is an error, not a warning.

2.10 Time Reference

All time indications are referenced to UTC. Unix epoch may be used for machine-readable times, [<u>ISO 8601</u>](https://www.iso8601.com/) for human-readable.

3. Command-Line Applications

3.1 Reference Implementation

The reference implementation is bac-common in bac-utils (lib/bac-common): make_parser and run_typer for the argument layer, the run error boundary, bac_common.ui for the terminal profile, bac_common.config for TOML settings and bac_common.testing for the standard-flag checks. lib/bac-common/tests/demo_tool.py is the shape of a tool. Every BAC tool is built on it; a tool that predates it is brought onto it when next touched. For presentation and interaction behavior, the BAC Interface Design Guide is authoritative; if the reference implementation differs, update the implementation rather than copying the deviation.

3.2 Stack

Component	**Library**
Language	Python 3.11+
Package management	uv – every tool ships a pyproject.toml compatible with uv tool install
Styled output	[<u>Rich</u>](https://github.com/Textualize/rich)
Argument parsing	argparse for simple tools; [<u>Typer</u>](https://typer.tiangolo.com/) for multi-subcommand tools
Interactive prompts	[<u>InquirerPy</u>](https://github.com/kazhala/InquirerPy) when guided selection genuinely helps
Config loading	tomllib (stdlib, Python 3.11+) for reading TOML configs
Secret loading	python-dotenv for .env files

3.3 Standard Flags

-v/--version and the hidden --debug are on every tool. The other flags appear where they apply: --dry-run wherever a run writes or changes something, --init and --config where the tool keeps persistent settings, --env-file where it reads secrets or machine-specific values, -l/--list where it works on a named set.

Flag	**Behavior**
-v / --version	Print toolname v0.x.x and exit.

| -l / --list | Where the tool operates on a named set of items (categories, targets, profiles), print a plain list of available items and exit. Only implement where it makes semantic sense. |

| --env-file PATH | Load a specific .env file instead of the default. |

| --dry-run | Plan and validate; do not perform any writes, uploads, or commits. Always print what would have happened. |

| --debug | Enable verbose output including raw tracebacks. Hidden from normal --help. |

| --init | Run first-time setup, or re-run it. Creates config files, prompts for required credentials, and validates the environment. If setup has not been completed and --init is not passed, the tool should detect this and exit with a clear message directing the user to run --init. |

The -v flag is non-negotiable. Every tool must have it.

Two conventions for tools that produce files: a filter writes to stdout unless -o FILE is given; a batch tool skips an output that already exists with a ! line and exit 0, --force redoes it, and two inputs that map to one output are an error before any work starts.

A tool that edits a user's files in place plans every change first, shows what it would change, and asks once before writing (--yes skips the prompt for scripts; --dry-run stops before it). Each written file keeps its previous content as \<file\>.bak beside it unless --no-backup; the .bak is the state before the most recent write and is replaced by the next one, so a copy that must survive several runs is the user's to make. Files are edited by span so that a diff shows only the intended change, and a file that changed between planning and writing is left alone and reported. A write that fails exits 1 after the summary.

3.4 Terminal Interface Implementation

Terminal presentation is specified in the BAC Interface Design Guide, Section 10, including Rich color markup, progress and spinner behavior, tables, inline bar charts, panels, headers, and the canonical run structure. Do not duplicate those rules here.

Implementation requirements in this guide are limited to the stack in Section 3.2 and the behavioral and packaging conventions below. Use Rich for the Interface Guide's terminal profile through bac_common.ui. Its colour constants – GREEN \#3FB950, RED \#E94538, YELLOW \#F7D400 – are generated from tokens.json by bac-design-tokens between marked lines in ui.py; use the constants, never retype the hex, and never edit the generated lines by hand.

3.5 Interactive Prompts

Use InquirerPy when a step benefits from guided input:

I```python
from InquirerPy import inquirer

Arrow-key selection
action = inquirer.select(
    message="Placement:",
    choices=["Accept", "Edit", "Skip"],
    default="Accept",
).execute()

Text field with pre-filled default
title = inquirer.text(
    message="Title:",

```
    default=decision.title,
).execute().strip()

# Confirmation
if not inquirer.confirm(message="Apply this plan?", default=False).execute():
    console.print("Aborted.")
    return

\
```

Criteria follow the BAC Interface Design Guide; in CLI implementation use these thresholds:

- The list has 4 or more items, or a wrong choice has a genuine cost such as a wrong upload target or a destructive action.
- The tool is running in an interactive terminal, not a CI pipeline.

For single yes/no confirmations that are low-stakes, a plain [y/N]: prompt is sufficient and avoids the InquirerPy dependency for minimal tools.

3.6 Configuration Files

Tools that have persistent settings (paths, API endpoints, preferences) should support a loadable and savable TOML configuration file. The config must be:

- Human-readable and hand-editable.
- Stored separately from source code, typically ~/.config/bac/{toolname}.toml or a project-local {toolname}.toml.
- Loadable via a --config PATH flag with a documented default path.

Config file structure:

I```toml # bac-toolname.toml # Build a CubeSat – toolname configuration

[source] repo_root = "~/repos/cubesat-resources" docs_path = "docs"

[upload] bucket = "cubesat-resources-prod" gcs_prefix = "resources"

```
\
```

Loading pattern:

I```python
import tomllib
from pathlib import Path

DEFAULT_CONFIG = Path.home() / ".config" / "bac" / "toolname.toml"

def load_config(path: Path | None = None) → dict:
    config_path = Path(path) if path else DEFAULT_CONFIG
    if not config_path.exists():
        return {}
    with open(config_path, "rb") as f:
        return tomllib.load(f)
```
```

\

Do not use YAML for config files. Do not commit config files containing live paths or credentials to the repository.

3.7 Secret Handling

Secrets (API keys, bucket credentials, tokens) are never hardcoded and never committed to the repository.

Preferred approach:

1. Store secrets in a .env file adjacent to the project root, and add .env to .gitignore.
2. Load with python-dotenv:

I```python from pathlib import Path from dotenv import load_dotenv

def load_env(env_file: Path | None = None) -> None: if env_file: load_dotenv(dotenv_path=env_file, override=True) else: load_dotenv() # searches current dir and parents

\

3. Expose a --env-file PATH flag, see Section 3.3, to let users specify a non-default .env.

4. Read secrets via os.environ or os.getenv after loading. Never pass raw secrets as function arguments across module boundaries.

Commit the .env template as .env.example without values:

I```text
Build a CubeSat – toolname secrets
Copy to .env and fill in values. Never commit .env.
GOOGLE_APPLICATION_CREDENTIALS=
GCS_BUCKET_NAME=
LLM_API_KEY=

\3.8 Error Handling and Exit Codes

Exit code	Meaning
0	Full success
1	Runtime error (catchall)
2	Bad arguments (standard argparse behavior)

- Print error messages to stderr, not stdout.
- Raise a BacError from bac_common.errors – or one of its subclasses, UsageError (exit 2), ConfigError, ExternalToolError – for clean user-facing errors; the run boundary prints the message and detail in the canonical rendering below and exits with the class's code. raise SystemExit("…") is the fallback for a tool without bac-common.
- Never print raw Python tracebacks to the user unless --debug is set.

- Error presentation follows the Interface Guide. Canonical Rich rendering:

Itext [#E94538]ERROR[/] Short description of what failed. Suggestion or detail, indented to align.

\3.9 --help Output

- Fits on one screen: 24 lines at 80 columns for the root help and for a subcommand with few options. A subcommand whose options are the tool's whole vocabulary may use up to 48 lines, with the examples still at the end and every option's help under about 45 characters so it stays on one line.
- One-sentence description of what the tool does.
- Flags listed with defaults noted.
- Two or three realistic usage examples at the end.
- Count the lines on Python 3.11: 3.13 prints -o, --out-dir DIR where 3.11 and 3.12 print the metavar twice and wrap the help onto a second line. bac_common.testing.assert_standard_flags checks the root help; a subcommand test that counts lines uses 48.
- Typer tools put the examples into the app's epilog with a \b line in front so Click keeps the line breaks; a subcommand's summary is cut at about 60 characters in the root help.

3.10 pyproject.toml Conventions

I```toml [project] name = "bac-toolname" version = "0.1.0" description = "One-line description." readme = "README.md" license = { text = "MIT" } requires-python = ">=3.11" dependencies = ["rich>=13.0", "python-dotenv>=1.0",]

[project.scripts] bac-toolname = "bac_toolname.cli:main"

[build-system] requires = ["hatchling"] build-backend = "hatchling.build"

\

- Tool names use the bac- prefix.

- Python package names use underscores (bac_toolname); CLI entry points use hyphens (bac-toolname).

- Always pin minimum version bounds on dependencies; avoid upper bounds.

- Add "InquirerPy\>=0.3" to dependencies if the tool uses interactive prompts.

4. Marimo Notebooks

4.1 Stack

Component	**Library**
Runtime	marimo (latest stable), managed via uv
Data manipulation	pandas for tabular work; polars for larger datasets

Tables	pyarrow\>=15.0 – required by mo.ui.table and mo.ui.data_editor; include it in every notebook that shows a table
Plotting (primary)	Altair – declarative, reactive-friendly
Plotting (secondary)	Matplotlib via mo.mpl.interactive() for specialized engineering plots
Plotting (3D only)	Plotly – only if interactive 3D visualization is genuinely needed
SQL	marimo built-in SQL cells for querying DataFrames or local databases

4.1.1 WebAssembly Notebooks (molab, mkdocs-marimo)

Some BAC notebooks run as WebAssembly notebooks instead of on a local Python install – notably molab-hosted notebooks and any notebook embedded via the mkdocs-marimo plugin on cubesat-resources.space. These execute under Pyodide, not CPython, which changes the stack constraints above:

- The notebook only has access to Pyodide's package set, at the versions Pyodide ships, not whatever is on PyPI. Compiled and native packages outside Pyodide's built list are unavailable, sgp4 among them; check Pyodide's package index before depending on anything with a compiled extension.

- PEP 723 inline script metadata, the \# /// script block, must declare dependencies with minimum version bounds only, for example pandas\>=2.0, never == pins, consistent with the dependency-pinning convention in Section 3.10. An exact pin that does not match what Pyodide has will fail to resolve.

- Molab keeps the PEP 723 bounds as written; it does not rewrite them (decided 2026-09-14). A notebook that comes back carrying == pins got them somewhere else, so look at the editing history rather than the platform.

- Memory is capped at 2 GB per notebook session. Keep dataset sizes and intermediate DataFrames within that budget; page or downsample large telemetry logs rather than loading them whole.

A model that only exists as a compiled package – NRLMSIS 2.1 through pymsis is the example – is evaluated offline by a script beside the notebook (scripts/build_density_table.py in the orbital lifetime notebook) and embedded in the notebook as a text table that a few lines of NumPy interpolate. The script carries its own PEP 723 block, so uv run resolves the compiled dependency where it exists, and its output is pasted over the data cell's string. A data block is never edited by hand; the cell's comment names the script that writes it, and a test compares the embedded table against a fresh evaluation at a few points. External data with a live source (the NOAA solar-cycle prediction) follows the same pattern as a snapshot with its fetch date, and a live fetch is an opt-in switch that ships off.

Local, non-WASM marimo notebooks, run via uv on a normal Python install, are not subject to these constraints and may use the full stack in the table above.

4.2 Notebook Structure

The header cell is a mo.vstack of the intro and a warn callout, not a single mo.md. The intro is three short paragraphs: what the tool estimates and how to drive it, meaning the panel or a profile and which siblings' profiles load; what to compare; and a links paragraph naming the tool's own bac.page URL, the sibling tools' URLs, buildacubesat.space and the bac-utils repository. The preliminary warning sits under the intro as mo.callout(kind="warn", title="Planning Estimates Only"). The ! line convention is retired for intros; ! stays a glyph for lines inside exported reports.

The reference cell order for the BAC notebook family:

1. Header – the mo.vstack described above.

2. Style – the cell in Section 4.3. One cell, near the top.

3. Imports – all imports, no side effects.

4. Constants and libraries – the fixed data the tool computes against.

5. Shipped profiles – the dedented module strings offered as downloads, see Section 4.7.

6. Profile loader – three cells: the upload element alone, then the parse cell, then the status block.

7. Control-panel elements – every mo.ui element the user drives, built through the clamping helpers in Section 4.4.

8. Panel layout – the two-column arrangement.

9. Sidebar – its own cell.

10. Panel render – the in-flow panel, or the one-line note that replaces it while the sidebar is on.

11. Resolve – one cell that reads the control elements and settles the parameter set everything downstream uses.

12. Computation cells – one logical responsibility each. A notebook that reads input files does it here, separated from transformation logic.

13. Chart conventions – the shared palette and styling helpers, see Section 4.5.

14. Headline cards – the mo.stat cards, with the cross-tool callouts under them.

15. Model callouts – the warn, danger, info and success readings of the result.

16. Map – where the tool has one.

17. Charts.

18. Tables.

19. Export – report, profile and CSV downloads, see Section 4.7.

20. Assumptions and acknowledgment.

21. Revision history.

Two constraints force the splits in that order: a cell cannot read a UI element it defines, which is what separates items 7, 8 and 10 and what forces the loader into three cells; and mo.sidebar must be the last expression of its own cell, which is why item 9 stands alone.

Do not mix computation and display in the same cell unless the display is trivially derived.

Position in the file is not execution order. The export cell reads ASSUMPTIONS_MD and ACKNOWLEDGMENT_MD, which are defined later in the file, and marimo resolves that because it orders cells by dataflow. It is correct as written and should not be "fixed" by moving the definitions up.

4.3 Styling and Identity Integration

Visual behavior is governed by the BAC Identity Guide and the BAC Interface Design Guide. Marimo

notebooks keep a dedicated style cell near the top so those rules are applied consistently. The cell below is the canonical text, identical across the notebooks. It is written by bac-design-tokens from tokens.json in bac-utils, the executable copy of the Identity Guide's tokens: do not edit it by hand – change the tokens and regenerate. A notebook that needs extra rules of its own (bac-pricing's .bac-sub and .bac-tg-lab) adds them after the cell's last line, where the generator leaves them alone.

Two corrections in it are easy to lose and each carries a comment in the cell. First, marimo styles its own markdown headings through a --heading-font custom property, and its .markdown h1, .markdown h2, ... selector outranks a bare h1, h2, h3, h4 rule, so without --heading-font set explicitly the headings render in marimo's default serif, Lora. Second, the dark tokens are not gated behind @media (prefers-color-scheme: dark): marimo's light and dark setting is independent of the OS theme, so the media query applies dark tokens against a light surface whenever the two disagree, which is unreadable. CSS light-dark() resolves against the color-scheme marimo itself sets, is Baseline 2024, and is what marimo uses internally. The cell carries no .vega-embed rule: the charts render on canvas inside a shadow root that no notebook stylesheet reaches, and chart text takes marimo's own chart theme (Section 4.5).

Typography follows the Identity Guide: Nunito Sans for headings (500) and prose (400), IBM Plex Mono with tabular figures for code and figures – never for body text.

```python
mo.md("""
<style>
@import url('https://fonts.googleapis.com/css2?
family=Nunito+Sans:opsz,wght@6..12,400;6..12,500;6..12,600;6..12,700&display=swap');
@import url('https://fonts.googleapis.com/css2?
family=IBM+Plex+Mono:wght@400;500;600;700&display=swap');

/* Written by bac-design-tokens from tokens.json – change the tokens, not this cell. */
:root {
  --text:       #3F3F3F;
  --bg:         #F9F9F9;
  --text-muted: #6D6D6D;
}

/* Track marimo's own light/dark setting rather than the operating system.
   marimo sets color-scheme from that setting and light-dark() follows it. */
:root {
  --text: light-dark(#3F3F3F, #E5E5E5);
  --bg:   light-dark(#F9F9F9, #1F1E1D);
  --text-muted: light-dark(#6D6D6D, #A9A8A6);
}

h1, h2, h3, h4 {
  font-family: 'Nunito Sans', system-ui, sans-serif;
  font-weight: 500;
  color: var(--text);
}

body, p, li, .prose {
  font-family: 'Nunito Sans', system-ui, sans-serif;
  color: var(--text);
}

code, pre, kbd {
  font-family: 'IBM Plex Mono', ui-monospace, monospace;
  font-variant-numeric: tabular-nums;
}
```

```
.marimo-callout { border-radius: 10px; }

/* marimo styles markdown headings through --heading-font with a selector
   more specific than the rule above, which would leave them in its serif. */
:root { --heading-font: 'Nunito Sans', system-ui, sans-serif; }
</style>
""")
```

Both families load from Google Fonts, so no self-hosting is needed. If a font fails to load it falls back to system-ui, a geometric sans, via the stacks above. Never fall back to a serif. Marimo has its own light and dark toggle; this CSS layers on top of it rather than replacing it.

Headings and callout titles in the marimo tools are title case, with articles, conjunctions and prepositions lowercase: "Cells Do Not Fit the Module", "Day in the Life", "Headline Numbers". This is the documented exception to the Identity Guide's sentence case recorded in Section 2.5, decided for the marimo tools on 2026-09-10.

Titles are matter-of-fact and name what the section holds: "Power Budget Breakdown", not "Where the Watts Go".

Notebook copy uses US spelling – "gray", "behavior", "color", "modeled" – matching the CubeSat Resources site since 2026-09-03.

4.4 UI Elements

Need	Element
Numeric range	mo.ui.slider(start=..., stop=..., step=..., show_value=True)
One option from 4 or more	mo.ui.dropdown
One option from 2 or 3	mo.ui.radio
Multiple options	mo.ui.multiselect
Date or date range	mo.ui.date / mo.ui.date_range
Boolean toggle	mo.ui.switch, or mo.ui.checkbox for inline use
Free text	mo.ui.text(placeholder="...")
File upload	mo.ui.file – only when the notebook accepts external input
Editable rows	mo.ui.data_editor – validate every value before use, see below
Tabular display	mo.ui.table for sort and filter; mo.ui.dataframe for full exploration

Set label= on every element. Do not scatter mo.ui elements across unrelated cells; the control panel is the only place inputs live.

Panel layout defaults to two columns, mo.hstack([left, right], justify="start", gap=2, wrap=True, widths="equal"), with rarely-touched groups in mo.accordion and one short help line per group. Three columns collide and should not be attempted. A tool whose controls genuinely need another arrangement may deviate, but the two-column default is what the family uses and what a reader expects.

mo.ui.slider must always be created with show_value=True. The default is False, and a slider with no visible value next to the handle is unreadable – the user can see roughly where it sits but not what it is set to. Add include_input=True where typed entry, rather than only dragging, helps precision.

Clamp profile values to the control's range. mo.ui.slider and mo.ui.number refuse a value outside start/stop and take the whole panel cell down with them, so a sibling's 30-day span loaded into a 14-day slider is fatal rather than merely wrong. Build every numeric control through N(**kw) and S(**kw) helpers that clamp value into range and round it to an integer when the step is integral.

Validate mo.ui.data_editor values before use. Numbers may arrive as empty strings, text, NaN or infinity; a checkbox column on a freshly added row may hold NaN or a string; a free-text name column may repeat. Require finite numbers, read a checkbox through a tolerant _flag() helper, and where rows are looked up by name, skip rows that repeat a name and report the skip rather than silently taking one of them.

A mo.ui element's value cannot be read in the cell that defines it. Marimo's reactivity only exposes the value to downstream cells, so a control whose options depend on another control's value cannot be built as a second, sibling control cell; put the dependent options in the data the first control drives, for example a column in an editable mo.ui.table or mo.ui.dataframe.

Where two elements stay editable after a preset sets them – a camera preset that fills in sensor and lens – the dependent cell compares the preset dropdown's current value with what the loaded profile said, which distinguishes "as loaded" from "changed since loading". When the two agree the profile's explicit choices win; once the user has moved the preset, the preset wins.

The sidebar pattern is one mo.ui.switch("Controls in a sidebar") rendered in the flow, the sidebar in its own cell, and the in-flow panel replaced by a one-line note while the sidebar is on. All three tools do it identically.

Known limitation: mo.ui.number renders thousands with a comma, for example 12,059, not the Swiss apostrophe required by Section 2.6. There is no supported override for the widget's own display; note the discrepancy rather than trying to reformat it.

4.5 Charts and Maps

Altair, the primary:

- Do not override the theme globally. alt.themes.enable('default') is deprecated as of Altair 5.5 in favor of alt.theme.enable(...), singular theme – use the new spelling in new notebooks. A notebook that sets no global theme at all is also acceptable.
- Width: width='container' for full width; explicit pixel width only for side-by-side charts. width='container' does not propagate into alt.vconcat, so a concatenated chart built that way will not fill the container. Stack charts as separate outputs, separate cells or mo.vstack, instead of alt.vconcat when full width matters.
- Chart text, axis lines, ticks, titles and subtitles carry no colour of their own: marimo hands vega-embed its dark theme on a dark surface – also under the "system" setting, which mo.app_meta().theme reports as light – and Vega's defaults apply on a light one. Do not set labelColor, titleColor, domainColor, tickColor or subtitleColor; a test keeps them out.
- Color encoding follows the BAC Interface Design Guide, Section 8.3: notebooks cannot know their surface, so categorical series use the shared palette (--chart-shared-1 … -5), which reads at about 4:1 on light and dark; beyond five categories, tints of the five. The ops plot colours (--chart-1 … -6) are per theme and the nebula triad is for Mission

material, so neither belongs in a notebook. Sequential → a single-hue ramp of the shared blue ('blues' is acceptable); diverging → 'redblue' only when semantically appropriate.

- Altair's default data transformer caps at 5'000 rows. Downsample before charting – one point a minute for a day of ground track – rather than switching to VegaFusion, which is another Pyodide dependency.
- A log axis drops the whole chart when a series contains 0. Filter or offset the zeros before the scale is applied.
- Always set axis titles explicitly. Do not rely on raw column names.
- Add a title to every chart: .properties(title='...'). The title carries the legend semantics, for example "dashed is the target margin".
- No decorative gridlines. Minimal axis ticks, as specified by the Interface Design Guide.

Chart conventions live in one cell – PALETTE, style_chart, rule_x, rule_y, the tints and IBM Plex Mono for chart figures – copied into every tool so the family renders identically. Two of its lines are written by bac-design-tokens: PALETTE = ["#2E7ED8", "#7D7D7D", "#9D7800", "#0E903A", "#E4463E"], the shared palette, and MUTED = "#A9A8A6" if IS_DARK else "#6D6D6D", the Identity Guide's --text-muted pair, used on reference rules and their labels, which have no theme default; IS_DARK comes from mo.app_meta().theme and follows the explicit setting only.

Matplotlib, via mo.mpl.interactive:

- Use plt.style.use('seaborn-v0_8-whitegrid') for consistent appearance.
- Every figure must have a title and labeled axes with units.
- Export as SVG for reports; PNG for inline display.

When not to chart:

- Data that fits in a table of fewer than 20 rows: use mo.ui.table instead.
- A bar chart showing a single value is not a chart, it is a number. Use mo.stat() or a mo.md() callout.

4.5.1 Map Recipe

Tools that draw a map share one recipe, and carry its settings in the [map] profile table of Section 2.7.1:

- Tiles are CARTO rastertiles/light_nolabels or dark_nolabels, keyed. BAC's free key lives in the notebooks and is restricted at carto.com.
- Tiles are drawn as mark_image at fixed pixel positions with .scale(None) on both channels, under an Altair mercator projection whose scale is 256·2^z/(2π).
- Every layer sets clip=True. The canvas is a fixed 960 × 540 px. Zoom comes from a slider.
- A switch turns the tiles off, with the Natural Earth coastline as the fallback base.

- Legends use orient="bottom", direction="horizontal".
- The chart is rendered bare inside the mo.vstack, because mo.ui.altair_chart refuses geoshapes. The tile credit line goes under it.
- Split ground tracks and circles at the date line on |Δlon| > 180, never a signed test. A westward crossing, which is what a retrograde orbit gives, otherwise draws a line straight across the map.

4.6 Output and Display

- mo.callout() – important results or warnings that must not be skipped.
- mo.stat() – single key metrics such as total mass budget or remaining power margin.
- Narrative text: mo.md() with full prose sentences. No bullet-list explanatory text.
- mo.accordion() – wrap verbose intermediate output such as raw data previews and debug tables so users can inspect on demand without scrolling past it.

Headline cards stay gray and the callouts do the judging. Six mo.stat(bordered=True) cards sit under a "Headline Numbers" heading in rows of two or three, with no direction= coloring; the pass or fail reading is a callout below them. The rule is uniform across the family, so no tool gets colored cards.

Callout kinds carry fixed meanings: warn for a model limit or a failed check, danger for a hard impossibility such as a lens that does not cover the sensor, a spacecraft inside the focal length, or frames that exceed the link, info for a note, and success for the single all-clear. Titles are title case, per Section 4.3.

Cross-tool callouts are named "X Against Y" – "Payload Demand Against This Link", "Passes Against the Link Budget" – and appear only when the loaded profile carries the sibling's [results.<tool>] table.

4.7 Profiles, Exports and Reports

The file contract is in Section 2.7.1. What follows is how the marimo tools read and write it.

The TOML writer is hand-rolled, because Pyodide has no tomli_w. It escapes strings for backslash, quote, newline, carriage return and tab; unwraps numpy scalars; drops NaN and infinity rather than writing them, since neither is valid TOML; writes booleans lowercase; and emits the header comment as one block. Shipped profiles are dedented module strings offered as downloads.

The reader is a P() helper that takes dotted sections, coerces booleans, and returns a float where the default is an integer, plus a profile_warnings list that collects values that are not numbers so the status block can report them. PH() is the companion that distinguishes "absent" from "present but set to nothing".

Export conventions:

Artifact	Label	File name
Report	Download report (.md)	bac-<tool>-<qualifier>-<date>.md
Profile	Download profile (.toml)	bac-<tool>-profile-<date>.toml

Artifact	Label	File name
Table	Download … (.csv)	bac-<tool>-<qualifier>-<date>.csv

These are Section 2.7's {project}-{descriptor}-{YYYY-MM-DD} with the version inside the file rather than in the name.

The report header reads: Generated YYYY-MM-DD HH:MM UTC (Unix N) with BAC <Tool> <version>, bac.page/<slug>. Profile: <name>. That is ISO 8601 for the human and Unix epoch for the machine per Section 2.10, and the tool's own short link per Section 2.4.

CSV exports never run values through fmt_int. The Swiss apostrophe is a display convention only, per Section 2.6.

The assumptions text is bold-lead paragraphs, one per model area, then a Limitations paragraph, then one closing line naming the glossary, the sibling tools and the source repository: "Linked terms go to the CubeSat Resources glossary, bac.page/glossary. The sibling tools are …; a profile saved from either loads here … Source and issues: bac-utils." Glossary terms are deep-linked on first use to the canonical https://cubesat-resources.space/references/glossary/#slug URL, with Python-Markdown's slugs – lowercase, punctuation dropped, spaces to hyphens: #bit-error-rate-ber, #ebn0, #downlink-uplink. The short link appears once, as text. No bullet lists in the assumptions.

4.8 Notebook Tooling and Testing

Run marimo check --fix before committing a notebook. It rewrites every cell's signature from the code, expands the return tuples and bumps __generated_with. A notebook edited by hand or by a model without it ends up with cells that take and return the same names. The same command without --fix is the lint step in CI.

Headless testing uses app.run(), which returns (outputs, defs). defs carries every non-underscore name, and outputs are Html objects whose .text is the rendered HTML – str() is not. To test a profile headlessly, copy the file and replace ui_profile.contents() with the file's bytes; tests/harness.py does that for all three tools.

marimo export html --no-include-code executes the full dataflow graph and is the cheapest check that every Altair spec validates. The exported page needs cdn.jsdelivr.net to render, which a build sandbox may block.

marimo check and app.run() do not see a chart that renders empty: the spec builds and validates, and the browser draws nothing. Two causes met so far: a field name with a dot (F10.7 (sfu)), which Vega-Lite reads as nested access – rename the column and keep the dot in the axis title; and a layer whose data is filtered to nothing. Render each chart's spec with vl-convert (the Altair objects are reachable from the app.run() outputs) or through marimo export html once per change that touches a chart. Vega's format() has no apostrophe thousands: use labelExpr = replace(format(datum.value, ',.0f'), /,/g, "'") on axes and legends and a preformatted text column for tooltips. A configure_legend() after the chart-conventions cell's own does not merge; set alt.Legend(orient="bottom") on the encoding.

4.9 Notebooks in the Repository

Every engineering notebook lives in bac-utils under notebooks/bac-<name>/: the notebook as one file bac_<name>.py, a README.md with numbered headings and a version history, tests/ with the headless checks, examples/ with the shipped profiles as TOML files, and a

pyproject.toml that makes the folder a workspace member and lists only the test dependencies (no build system, no entry point); its version follows the notebook's TOOL_VERSION, and a test keeps the two equal. The notebook keeps its PEP 723 block with minimum bounds and cannot depend on bac-common, because its packages are not in Pyodide. Its first line is # SPDX-License-Identifier: MIT, above the PEP 723 block.

The repository is the source and the links read it: every published notebook has two bac.page links, bac.page/molab-<tool>-app (marimo.app in read mode with the code hidden – the one to use) and bac.page/molab-<tool> (the notebook in molab's editor), both opening the file from main on GitHub. A merge publishes; nothing is copied anywhere. The notebook's intro links both of its own and the app link of each sibling, and its closing section carries both for each sibling. A change that moves a number is a new version, with a row in the revision-history cell, re-recorded figures in the README's "Numbers at the defaults" and in the regression tests.

Notebooks are exempt from -v and --help; the version is TOOL_VERSION in the constants cell and appears in every export's header. They are formatted and linted by ruff like the rest of the repository, with E501 (markdown rows cannot wrap), B018 (a cell's last bare expression is its output) and F841 (a name assigned at the top of a cell is a notebook global) exempt. CI runs marimo check on every notebook and the headless tests through the root pytest.

5. Blender Extensions (VSE Tools Collection)

5.1 General Principles

- VSE extensions register under the category "VSE Tools" (bl_category = "VSE Tools"), placing them in the dedicated VSE Tools tab of the VSE N-panel. This follows Blender's own naming convention for the video editing context.
- Extensions intended for the 3D viewport or Properties panel, meaning non-VSE, use bl_category = "BAC" instead.
- UI labels use title case. This is a deliberate exception to the brand's general sentence-case rule, recorded in Section 2.5, made because Blender's native UI is title-cased and BAC panels should match their surroundings. No abbreviations unless they are standard Blender conventions such as "FPS" and "AVI".
- Operator buttons that perform a destructive or long-running action must include a confirm dialog or a visible before/after indicator.
- Long-running operators run as modal timers that pass every event but Esc and the timer through (PASS_THROUGH), implement cancel(), and keep their settings in a property group on the scene rather than on the operator. A job-backed operator such as Rebuild Proxy is called with "INVOKE_DEFAULT"; the plain Python call runs its synchronous exec and blocks the interface.

5.2 Sub-Panel Structure

Each tool registers as a named sub-panel. The VSE Tools collection (bac-vse-tools) has three:

IVSE Tools tab

└— Bulk Import (sub-panel)

└— Sequential Proxies (sub-panel)

└─ Strip Utilities (sub-panel)

\

Each sub-panel contains:

1. A single layout.label(text="...") line giving a plain-text description of what the operator does. One line; fits without wrapping.
2. One or more operator buttons, full-width, each with a Blender built-in icon.
3. Property rows below the buttons.

This order – description, button, properties – is fixed. Do not put properties above the operator button.

5.3 Operator Buttons

Full-width, single row. Icon on the left; descriptive text label. Technical constraints may appear in parentheses in the label text:

I```python row = layout.row() row.operator("sequencer.bac_bulk_import", text="Bulk Import VSEN", icon="IMPORT",)

row = layout.row() row.operator("sequencer.bac_sequential_proxies", text="Sequential Proxy Build (AVI only, 3-min timeout)", icon="SEQUENCE",)

\5.4 Property Rows

Properties are displayed as key-value rows directly below the operator button. Logically paired, short properties may share a single row:

I```python
Paired row
row = layout.row()
row.prop(props, "pause_seconds")   # displays as "Pause (s)" 0.10
row.prop(props, "retries_per_file")   # displays as "Retries per File" 2

Full-width single row
layout.prop(props, "min_length")   # displays as "Minimum Length (s)" 5.00

\

Values are right-aligned by Blender's default prop() rendering. Do not override this.

5.5 Property Naming

- bpy.props property names: snake_case, the internal identifier.
- Display names (name=): title case, with units in parentheses where applicable.
  - "pause_seconds" → name="Pause (s)"
  - "min_length" → name="Minimum Length (s)"
  - "retries_per_file" → name="Retries per File"

  - "stable_time" → name="Stable Time (s)"

- Do not expose raw snake_case identifiers as display names.
- description= is required for every property exposed in the UI. Write as a complete sentence without a trailing period.

5.6 Layout Details

- Use layout.separator() between the description label and the operator button only if there is visual crowding. Do not use it mechanically above every element.
- Icons: Blender built-in icon set only. Icons reinforce the label; always include text alongside the icon.
- The ::: panel options menu, Blender's standard bl_options = {'DEFAULT_CLOSED'} or similar, is available automatically on sub-panels. No special handling needed.

6. Documentation and Governance Conventions

- Governance guides are independently versioned. The BAC Identity Guide, BAC Interface Design Guide, and BAC Project & Tooling Guide carry their own version numbers because they govern multiple artifacts.
- Product, subsystem and software documentation follows its subject's version. A README or user guide names the version of the thing it documents and does not create an unrelated document version number.
- Every BAC documentation file includes a version history or revision history appropriate to its class. Governance guides use their own guide version; subject documentation records meaningful documentation changes while keeping the subject version clear. A history is the minimum traceability requirement for a project aiming at flight hardware.
- When updating a governance guide, add a brief entry at the top of its changelog with the date and a one-line description of what changed.
- Titles and headings are matter-of-fact and name their subject: "Power Budget Breakdown", not "Where the Watts Go".
- Use the punctuation, casing, and voice rules from the BAC Identity Guide for prose unless a host platform has an explicitly documented exception, of which there are two: Section 5.1 and Section 4.3.

7. Quick Reference – The Non-Negotiables

1. Hardware: CERN OHL-S 2.0. Software: MIT. Documentation: CC BY-SA 4.0. The BAC name and marks are excluded and follow the BAC Trademark Policy.
2. Physical hardware uses v<version>r<revision>[.<patch>][-<variant>]; software uses SemVer with v in display output.
3. Governance standards are independently versioned; product, subsystem and software docs follow the subject they document.
4. Use Build a CubeSat on first display reference and BAC thereafter; lowercase bac is for code, filenames and identifiers.

5. Prefer bac.page short links for user-facing docs, products and releases; use canonical URLs for APIs and machine endpoints.

6. Human-edited config is TOML; tabular exchange and logs are CSV; machine-to-machine is JSON; avoid YAML unless a framework requires it.

7. Codeberg is the primary source forge; GitHub hosts the three repos that need its Pages and Actions: CubeSat-Resources, bac-page and bac-utils.

8. All times are UTC; Unix epoch for machine-readable time, ISO 8601 for human-readable time.

9. Python tools use uv, build on bac-common, include -v/--version, support -l/--list where meaningful, keep secrets in .env loaded via python-dotenv, and when they edit files in place plan, show, confirm and leave a .bak.

10. Interface behavior comes from the BAC Interface Design Guide; identity tokens and Ops/Mission mode come from the BAC Identity Guide. Copies of the tokens in code are generated from tokens.json by bac-design-tokens, never edited by hand.

11. Marimo and Blender implementation conventions in this guide are authoritative for BAC tooling on those platforms.

12. Marimo tools share the mission profile contract in Section 2.7.1; a tool loads a sibling's profile, takes what it owns, and ignores the rest.

13. Fail with actionable errors, validate before writes and API calls, and expose --dry-run where changes can be previewed.

8. Version History

Version	Date	Change
0.15	2026-10-27	The design rebase of Identity Guide 1.0 and Interface Design Guide 1.0 carried into tooling. Section 2.5: tokens.json in bac-utils is the executable token source; bac-design-tokens writes the gallery's and the bac-suite's tokens.css, the Rich colours and the marimo style cells, --check fails CI on a stale copy, and other repositories such as BAC Ground take a copy of the generated tokens.css. Section 2.8: the bac-utils row names the design tokens. Section 3.4: bac_common.ui's colours are generated – #3FB950, #E94538, #F7D400. Section 3.8: the error rendering in the new red. Section 4.3: the canonical style cell rewritten – #F9F9F9/#1F1E1D, #3F3F3F/#E5E5E5, #6D6D6D/#A9A8A6, Nunito Sans body and 500 headings, IBM Plex Mono for code only, 10px callouts – generated rather than copied; the stale third correction about .vega-embed removed (the rule left the notebooks on 2026-10-24). Section 4.5: chart text takes marimo's chart theme; the shared notebook palette and the new MUTED pair, both generated. Section 7: item 10 names the generated copies. Section 2.1 and Section 7: the BAC Marks are excluded from the open licences and follow the new BAC Trademark Policy 1.0; repositories that ship logo artwork say so in their README. Section 2.8: the guides move to github.com/buildacubesat/bac-

Version	Date	Change
		design.
0.14	2026-10-24	The bac-utils homogenization written down. Section 2.3: the attribution rule for company, project and mixed fields (decided 2026-10-24). Section 2.7.1: the shared [spacecraft] and [[deployables]] tables, directions in letter form in the owning tool's table, panels matched to deployables by name (decided 2026-10-24). Section 2.2: git tags <tool>-v<X.Y.Z> where another repository pins a version; fast-forward merges. Section 2.6: lowercase axis letters in command-line values and file names. Section 2.7.1: the 500 km planning orbit. Section 2.8: the bac-utils row; bac.page renamed bac-page. Section 3.1: bac-common is the reference implementation. Section 3.3: -v and --debug universal, the other flags where they apply; stdout and -o for filters; skip-existing and --force for batch tools; the in-place editing rule (plan, show, confirm, .bak, span edits). Section 3.8: BacError and its subclasses instead of SystemExit. Section 3.9: 48 lines for a subcommand with many options; count on Python 3.11; Typer epilogs. Section 4.1.1: compiled models as offline tables. Section 4.3: the style cell's values copy the gallery's tokens.css; --text-muted is #6C6B67/#A3A29C. Section 4.5: MUTED follows. Section 4.8: what the headless checks miss. New Section 4.9: notebooks in the repository – folder layout, the two bac.page links reading main, versioning, ruff exemptions. Section 5: the third sub-panel, modal operators, the extension's own labels and operator ids; the known deviation is gone.
0.13	2026-09-15	Homogenization pass over the three marimo notebooks (link budget 0.7.0, optical payload 0.7.0, power budget 0.5.0), plus a presentation pass over the whole guide. Guide-wide: headings to title case, US spelling, section references spelled out rather than using the paragraph sign, markdown escape artifacts and stray hard line breaks removed, bold reduced. Section 2.5: recorded the marimo title-case exception beside the Blender one. Section 2.6: axis directions never use sign notation; the thousands separator is display only. Section 2.7: added 2.7.1, the shared TOML mission profile contract, with normative key names for [orbit], [ground_station], [target] and [map], the [results.<tool>] convention, per-row results and author detection. Section 4.1: pyarrow>=15.0 for notebooks that show tables. Section 4.1.1: molab keeps PEP 723 bounds as written. Section 4.2: the header-cell recipe and the 21-step reference cell order, the two constraints that force the splits, and the dataflow-ordering note for the export cell. Section 4.3: the canonical style cell including the .vega-embed rule and comments for all three corrections; title case; matter-of-fact titles; US spelling. Section 4.4: clamping helpers, data_editor validation, preset versus profile resolution, the sidebar pattern, the two-column panel default and the mo.ui.number separator limitation. Section 4.5: the shared chart-conventions cell, the 5'000-row transformer cap, the log-axis-with-zero failure, and 4.5.1, the map recipe with the date-line split. Section 4.6: gray headline cards with

Version	Date	Change
		callouts doing the judging, fixed callout kinds, cross-tool callout naming. New Section 4.7 for profiles, exports and reports. New Section 4.8 for notebook tooling and testing.
0.12	2026-09-08	Five marimo corrections found while building the BAC Link Budget notebook. Section 4.3: replaced the fallback style cell's @media (prefers-color-scheme: dark) block with CSS light-dark(), because marimo's light and dark setting is independent of the OS and the media query could leave light text on a light surface; added --heading-font, without which marimo's .markdown h1 selector wins and headings fall back to Lora. Section 4.5: replaced the deprecated alt.themes.enable('default') with alt.theme.enable(...); noted that width='container' does not propagate into alt.vconcat; added the .vega-embed text fill rule. Section 4.1: added 4.1.1 on WebAssembly notebooks – Pyodide's fixed package set and versions, minimum-bound-only PEP 723 dependencies, unavailable compiled packages such as sgp4, and the 2 GB memory limit. Section 4.4: mo.ui.slider must set show_value=True; a mo.ui value cannot be read in its own defining cell; mo.ui.number renders thousands with a comma.
0.11	2026-08-10	Renamed the former BAC Internal Tools Design Language Guide to BAC Project & Tooling Guide and reorganized it under the new three-guide architecture. Removed visual and interface authority from this guide in favor of the BAC Identity and Interface Design guides while retaining platform-specific implementation recipes. Clarified the documentation-versioning rule: governance standards are independently versioned; documentation tied to a versioned artifact follows that artifact's version.
0.10	2026-07-08	Clarified the thousands separator: ' for CHF and metric, comma for USD prices. Mirrored in the design system.
0.9	2026-07-08	Reconciled with the new BAC Design System Specification. Retired Moon 2.0 → Nunito (wordmark), Nunito Sans (headings), IBM Plex Mono (body); removed self-hosted font URLs. Removed amber – warnings now use BAC Yellow. Aligned dark-mode token names to the design system (--text, --bg, --text-muted) and updated the marimo style cell. Added a relationship note establishing the authority split, a radius-scale pointer, a nebula charting option, and a Blender title-case exception note. Fixed a duplicate section number and SKU typos.
0.8	2026-06-04	Added the liveness requirement to terminal progress and feedback: tools must always show a spinner or progress bar during any silent operation longer than about 1 second.
0.7	2026-06-04	Added the --init flag to the standard flags.
0.6	2026-06-04	Added SKU conventions for tools interacting with Shopify, inventory, or the production pipeline.
0.5	2026-06-04	Added versioning: the hardware version and revision scheme, SemVer for software, and the doc titling convention. Updated Moon 2.0 font references to BAC GCS URLs with all three weights;

Version	Date	Change
		renumbered the project-wide sections.
0.4	2026-06-04	Filled in the Blender and VSE section from a screenshot: sub-panel structure, operator pattern, property rows, naming. Added repository locations for the Codeberg and GitHub split; bumped status colors again; added the documentation versioning requirement.
0.3	2026-06-04	Naming: lowercase bac is fine in code and filenames; no mixed case. Added dark mode: system default, dark fallback, CSS variables. Updated off-white and off-black to #efefed and #201e1c. Bumped status colors slightly. Moved No Emojis to the last core value. Removed the ASCII art logo. Fixed pyproject.toml field order and removed inline comments. Updated the marimo style cell with dark mode CSS.
0.2	2026-06-04	Major update: added the project-wide conventions section covering licensing, branding, palette, typography, shapes and data formats; updated the CLI stack to InquirerPy; added standard flags, config files, and secrets sections; corrected the table box style to match the reference implementation; updated the color palette with muted warm hex values.
0.1	2026-05-22	Initial draft. CLI and marimo sections complete; Blender section a stub.